

altairsim — The Debugger

2026-09-20 · 5bffb26

A simulation of the MITS Altair 8800 and the S-100 bus

Debugging	1
Where the processor is — REGS	1
Stepping — STEP , NEXT	2
Breakpoints — BREAK , NOBREAK	2
Reading a block of memory — DUMP	5
One byte at a time — EXAMINE , DEPOSIT , EDIT	5
Disassembling — DISASM	6
Symbols — SYMBOLS , SHOW SYMBOLS	8
Searching, filling, moving — SEARCH , FILL , MOVE , COMPARE	10
Running real bus cycles by hand — IN , OUT	10
Asking without touching — WHO	11
Looking at the bus itself — SHOW BUS	11
The machine over time — TRACE , HISTORY	12
What a board is doing — SET ... DEBUG , SHOW DEBUG	14
A copy of the session — SET CONSOLE log	16
A debugging session	16
Saving state — SNAPSHOT , RESTORE	18
Getting an assistant to do it — the MCP server	18
Things to know about the bus	20

Debugging

This is the document the simulator exists for.

It is the companion to *The Monitor*: that document introduces the `altairsim>` prompt — how you start, stop and reconfigure the machine — and this one covers what you do at that prompt when something has gone wrong. Both stand beside the *User Manual*, which describes the emulated hardware. `altairsim` simulates the MITS Altair 8800 and the S-100 bus; if the `altairsim>` prompt is new to you, read *The Monitor* first.

Running old software is the easy half. The hard half is being able to see what a machine is actually *doing* — which board answered, what went out on the bus, why the interrupt never arrived — and that is what the commands here are for. Not every monitor command is one of them — these are the ones you reach for when something has gone wrong.

Where the processor is — REGS

`REGS` prints the whole processor on one line. The flags come first — carry, zero, minus, even parity, interdigit carry — then the register pairs, the stack pointer, the interrupt-enable flip-flop, and the program counter. The last column is **the instruction the processor is about to execute**, already disassembled.

You get this line free every time the machine stops, so most of the time you never type `REGS` at all.

```
altairsim> REGS
C0Z1M0E1I0 A=00 BC=007F DE=CA01 HL=BC0E SP=BC37 IE=1 PC=CA9C  CALL CA78
```

Each register is labelled the way you would name it to `SET REG` — the pairs `BC / DE / HL`, the stack pointer `SP`, the program counter `PC` — so what you read is what you type back.

The line follows the CPU in the machine, so it shows exactly the registers that processor has. A Z80 has more to show — extra flags (`S`, `P`, `H`, `N` beside the shared `C` and `Z`), the whole alternate register bank and its own flags, the `IX` and `IY` index registers, and the interrupt vector and mode `I / IM` — too many to sit on one line, so it wraps to two. The **first** line carries the registers it shares with the 8080 — the flags, `A`, the pairs, `SP`, `PC` — with the interrupt-enable flip-flop shown as `IFF1` (the Z80's own name for it, where the 8080 says `IE`), and the instruction the `PC` points at, decoded in the Z80's own mnemonics. The **second** line is everything the Z80 adds — the shadow flags, the alternate bank `A' / BC' / DE' / HL'`

(a prime marks each), then **IX**, **IY**, the interrupt vector **I**, the mode **IM**, and **IFF2** (the shadow of the **IFF1** on the first line):

```
altairsim> REGS          (on a Z80 machine)
C0Z0S0P0H0N0 A=00 BC=0000 DE=0000 HL=0000 SP=0000 IFF1=0 PC=0000 CALL PE,9A78
C0Z0S0P0H0N0 A'00 BC'0000 DE'0000 HL'0000 IX=0000 IY=0000 I=00 IM=00 IFF2=0
```

Flags are registers, and you may set them:

```
SET REG A=3F
SET REG CY=1
```

Stepping — **STEP**, **NEXT**

STEP runs **real bus cycles through the real instruction decode**. It is not an interpreter running alongside the machine — it *is* the machine, moved forward by one instruction. It prints one line per instruction — the machine *after* that instruction ran, with the one the PC has now reached — so **STEP 3** shows three lines, one for each step. Past thirty-two it runs quietly and tells you where it ended up.

```
STEP          one instruction
STEP 20       twenty of them (a count, so it is decimal)
```

NEXT **steps over a subroutine**. At a **CALL** or **RST**, **STEP** walks you down into the callee — every instruction it runs, and everything it in turn calls. Often you do not care: the routine works, and you want the *next* instruction in the code you are reading, not a tour of a print routine. **NEXT** gives you that. On a **CALL** or **RST** it runs the callee at full speed and stops the instant it returns; on anything else it is just a single step. It does exactly what you would do by hand — sets a breakpoint at the return address and runs to it — so the callee is live while it runs: it can read the console, and **^E** (STOP) or **^C** stops it if it never comes back. A breakpoint that fires *inside* the callee stops you there, as it should.

```
NEXT          over the CALL/RST at PC, else one instruction
N             the same -- it owns the letter, because you type it constantly
```

Breakpoints — **BREAK**, **NOBREAK**

There are three kinds, and only the first is about the processor at all.

BREAK MEM and **BREAK IO** watch bus cycles, not instructions. That is a much stronger thing, and it is the reason to prefer them. A memory watch will catch a DMA transfer that no instruction on the CPU ever

performed, and it will work unchanged on any processor you put in the machine, because it is watching the backplane rather than the program.

If you are chasing a byte that keeps getting clobbered, `BREAK MEM W <addr>` will find who is doing it, whatever is doing it.

A cycle watch stops *before* the access, with the PC on the instruction that was about to make it — nothing executed, no port read, no byte written, every register as it stood. It is the same place a plain `BREAK <addr>` stops, so `RUN` or `STEP` runs that instruction fresh. (The one exception is a cycle a *DMA* board drove: that has no CPU instruction to hold back, so it stops at the instruction boundary after the transfer instead.)

BREAK TAPE STOP watches a device, not the program at all. It stops the machine the moment a cassette deck reaches auto-stop — the instant the tape parks itself after a load has finished feeding. That is exactly when you want to look at what landed, and you get there without having to know the loader's end address: arm it, run, and the machine halts inside the loader the moment the tape stops.

```
BREAK FF13      stop when the PC gets there
BREAK 2C00-2CFF  ...or anywhere in a range
BREAK MEM W 100  stop when ANYTHING writes to 0100
BREAK IO  R 10   stop on an IN from port 10
BREAK TAPE STOP  stop when a cassette deck auto-stops after a load
BREAK           list them
NOBREAK 2        clear one (the id is a plain decimal, not a bus address)
NOBREAK         clear them all
```

Breakpoint ids are handed out in order — 1, 2, 3 — and restart at 1 once no breakpoints are left, whether you cleared them all with `NOBREAK` or removed the last one by id. Removing a breakpoint from the middle does not renumber the rest, so an id is not the running count of live breakpoints.

When one fires, it says which, where, and what the machine was doing. A plain address breakpoint stops with the PC *on* the instruction it names — nothing there has run yet:

```
altairsim> BREAK 2C00
breakpoint 1: pc      2C00
altairsim> RUN FF00
breakpoint 1 (pc      2C00) -- stopped at 2C00
1414 instructions, 9202 T-states.
C0Z1M0E1I1 A=C9 BC=0000 DE=2CEB HL=FFFE SP=0000 IE=0 PC=2C00 DI
```

The header names the breakpoint that fired and where the machine stopped, then how far it ran since `RUN`, then the register line `REGS` would print — flags, registers, `PC` (the program counter), and the instruction about to run. A cycle watch reads the same way, but *stopped at* is the instruction that made the

access, not the watched address — so it hands you the culprit. Here a write to `2C00` is caught with the PC on the `STAX D` that wrote it:

```
altairsim> BREAK MEM W 2C00
breakpoint 1: mem w 2C00
altairsim> RUN FF00
breakpoint 1 (mem w 2C00) -- stopped at FF09
4 instructions, 34 T-states.
C0Z0M0E0I0 A=F3 BC=00EB DE=2C00 HL=FF13 SP=0000 IE=0 PC=FF09 STAX D
```

An address breakpoint can carry a condition. `BREAK <addr> IF <expr>` stops only when the expression is true — the registers, tested the moment the PC reaches the address. It is what you reach for when a breakpoint fires ten thousand times before the once you care about: put the distinguishing state in the condition and let the machine run until it holds.

A bare word that names a register *is* that register, so a literal needs a leading zero — `0A` is ten, `A` is the accumulator. `==` `!=` `<` `>` `<=` `>=` compare, `&&` `||` combine, `&` `|` mask, and parentheses group.

```
BREAK 100 IF A==0
BREAK 100 IF HL==8000 && Z==1
BREAK 100 IF (A&0F)==0          only when the low nibble is zero
```

A cycle watch can carry a condition too. `BREAK MEM W 100 IF <expr>` or `BREAK IO R 10 IF <expr>` stops only on the access whose registers satisfy the condition — so you can wait for the *one* write to a shared buffer that happens while a particular counter holds, or the read of a status port taken with a specific unit selected. The registers `IF` tests here are the ones the instruction ran *with* — its inputs, as they stood the moment it began — the same state a `BREAK <addr> IF` at that instruction would see.

```
BREAK MEM W 100 IF B==0          the write to 0100 taken with B already zero
BREAK IO R 10 IF C==1            the IN from port 10 while unit 1 is selected
```

LOADS tests what an IN read. An `IF` on a port read sees the registers *before* the instruction, so it cannot ask about the byte the `IN` just fetched — that byte is not in any register yet. `BREAK IO R <port> LOADS <expr>` is the other half: it judges the condition *after* the instruction retires, so the register the `IN` loaded holds its new value. It is how you stop on the *content* of a port rather than the fact of a read — the status bit that finally came up, the byte that was out of range.

```
BREAK IO R 10 LOADS A>7F          stop when the IN from port 10 reads a high byte
BREAK IO R 08 LOADS (A&80)!=0    ...when bit 7 of the status port is finally set
```

`IF` and `LOADS` are the same expression read at opposite ends of the instruction: `BREAK IO R 10 IF A==5` is about the `A` that went *in*, `BREAK IO R 10 LOADS A==5` about the `A` that came *out*. `LOADS` belongs to a port read — a write and a memory cycle load no register — so it is only accepted on `BREAK IO R`.

Reading a block of memory — DUMP

DUMP is how you read a lot of memory at once. A bare **DUMP <addr>** runs to the **end of its page**, and a bare **DUMP** carries on from there — so however you first landed, the rows stay page-aligned and the columns never move under your eye.

It only *looks*. Nothing is consumed and no bus cycle is run.

```
DUMP 100          0100-01FF: a whole page
DUMP              the next page
DUMP FF00-FF0F    exactly that range
DUMP 100/20       0100-011F (a length, and it is part of the address, so it is hex)
DUMP 0 WIDTH=8    eight bytes to a line (a count: decimal)
```

Each row is the address, then the bytes in hex, then the same bytes as text — a byte that is not a printable character shows as `.`. Here is the default machine's DBL boot PROM at **FF00**:

```
altairsim> DUMP FF00-FF3F
FF00  21 13 FF 11 00 2C 0E EB  7E 12 23 13 0D C2 08 FF  !.....~.#.....
FF10  C3 00 2C F3 AF D3 22 2F  D3 23 3E 2C D3 22 3E 03  ....."/.#>,">.
FF20  D3 10 DB FF E6 10 0F 0F  C6 10 D3 10 31 79 2D AF  .....1y-.
FF30  D3 08 DB 08 E6 08 C2 1C  2C 3E 04 D3 09 C3 38 2C  ....>....8,
```

This is code, so the text column is mostly `.` — it earns its keep on a buffer of strings, where you can read the message straight out of the right-hand column.

One byte at a time — EXAMINE , DEPOSIT , EDIT

These are **the front panel's switches**, and they behave like them. **EXAMINE** shows a single byte — hex, ASCII, and its bits. It also jams the address into the program counter, exactly as the switch does, so **EXAMINE <addr>** follows the byte with the register line and the disassembled instruction the PC now points at — what the next **STEP** will run. A bare **EXAMINE** steps to the next byte, quietly, which is the panel's **EXAMINE NEXT**.

DEPOSIT runs a **real bus write**. If no board decodes that address, it says so rather than pretending to have stored something — which is the difference between a debugger and a notepad.

```
EXAMINE 2C00      one byte: hex, ASCII, and its bits – then the register line
                  and the instruction the PC now points at, ready for STEP
EXAMINE          the next byte, quietly – the panel's EXAMINE NEXT
DEPOSIT 100 C3 00 2C
```

`EDIT` is `DEPOSIT` done interactively — the way you patch a run of bytes without retyping the address each time. The prompt shows an address and the byte that is there; type a new value and Enter writes it and drops to the next byte, a bare Enter leaves the byte untouched and drops to the next, and `.` returns you to the monitor. It is the same real bus write, so it warns the same way when nothing decodes the address, and `EDIT <addr> ROM` burns a PROM. `EDIT` needs a keyboard — at the prompt or down a pipe — so where there is none (an automated `startup` list) reach for `DEPOSIT` instead.

```

EDIT 100      0100 C3 3E      type 3E, Enter – written, on to 0101
                0101 00      Enter alone – left as 00, on to 0102
                0102 2C .      a '.' stops and returns to the monitor

```

On a machine with a CPU, `EDIT` will also take an **instruction** where a byte would go and assemble it in place — `EDIT` is `DISASM` (below) read the other way. Type `IN 10` and it writes `DB 10`; the prompt then drops by the instruction's length, not one byte, so a two-byte instruction lands the next prompt two on. Operands are numbers in the console base — an `H` or `Q` suffix on the number overrides it — and there are no labels: this is a patch assembler, not a toolchain. A bare value is still a plain byte, so byte entry is unchanged.

The 8080 and 8085 assemble in full. The Z80 assembles as a convenience — its documented main, `CB` and `ED` instructions — but not the `IX`/`IY` indexed forms or the relative jumps `JR` and `DJNZ`, which report *not implemented* rather than take a byte. Those two ask for something a single prompt cannot supply: an indexed form carries a displacement (and its `IXH`/`IXL` half-registers are undocumented besides), and a relative jump encodes a signed offset from the address *after* it, so its byte depends on both where you are jumping to and where you are jumping from. Deposit those bytes directly, or reach the same place with `JP`. A CPU whose encoding is not assembled here at all keeps taking bytes.

```

EDIT 100      0100 C3 IN 10      assembles DB 10, on to 0102
                0102 00 LXI H,FF13 assembles 21 13 FF, on to 0105
                0105 76 .        '.' returns to the monitor

```

Disassembling — `DISASM`

`DISASM` **peeks**: it reads memory without running a bus cycle. That matters, and it is not a detail. A `read()` on a serial board *consumes* a byte from its receiver, and a disassembler that ate the guest's input while you were looking at it would be a debugger you could not trust. Nothing here that only *looks* at memory will disturb it.

```

DISASM FF00      sixteen instructions
DISASM          carry on
DISASM 0-2F      exactly that range

```

A worked example — ALTMON's reset entry at F800 , the first thing the ROM runs:

```
altairsim> DISASM F800-F811
F800 3E 03    MVI A,03
F802 D3 10    OUT 10
F804 D3 12    OUT 12
F806 3E 11    MVI A,11
F808 D3 10    OUT 10
F80A D3 12    OUT 12
F80C 31 00 C0 LXI SP,C000
F80F CD A5 FB CALL FBA5
```

It resets both 2SIO channels' 6850s (OUT 10 / OUT 12), selects 8N2 (MVI A,11), points the stack at C000 , and calls the sign-on routine at FBA5 . Stopping at F811 is deliberate: the bytes that follow are the sign-on text, and DISASM would decode that ASCII as instructions — nothing in memory says which bytes are code.

DISASM trusts you to start on an opcode, and it cannot check. Give it an address in the *middle* of an instruction and it will decode the operand bytes as if they were opcodes, and the listing it prints is fiction. Start the same reset code one byte late, at F801 , and the 03 that was the *operand* of MVI A,03 becomes an instruction in its own right:

```
altairsim> DISASM F801-F812
F801 03      INX B
F802 D3 10    OUT 10
F804 D3 12    OUT 12
...
```

INX B is a phantom — there is no such instruction in this ROM. A disassembler usually re-syncs after a byte or two (here F802 is back on the real code, because 03 happens to be one byte long), so a listing can look right a few lines down while its first instruction is nonsense. When a DISASM reads oddly, check that you started where an instruction *starts*: single-step to the address with STEP , or begin the range at a label you trust.

DISASM decodes for the CPU the machine is running. The same bytes are different instructions on different processors, so DISASM reads them through whichever CPU is in the machine — and prints in that

CPU's own assembly dialect. Put a Z80 in the machine and bytes that are undefined on an 8080 become real instructions, in Zilog mnemonics (`LD` , not the 8080's `MVI / MOV`):

```
altairsim> DISASM 100-10A          (on a Z80 machine)
0100 ED B0      LDIR
0102 CB 27      SLA A
0104 18 FE      JR 0104
0106 10 FC      DJNZ 0104
0108 DD 7E 05   LD A,(IX+05)
```

The very same bytes on an 8080 have no `ED` , `CB` or `DD` prefix to give them meaning, so it flags each undefined byte (`??=`) and decodes what is left as unrelated 8080 instructions:

```
altairsim> DISASM 100-10A          (the same bytes, on an 8080 machine)
0100 ED        ??= ED  *CALL
0101 B0        ORA B
0102 CB        ??= CB  *JMP
0103 27        DAA
0104 18        ??= 18  *NOP
0105 FE 10     CPI 10
0107 FC DD 7E  CM 7EDD
010A 05        DCR B
```

Symbols — `SYMBOLS` , `SHOW SYMBOLS`

Everything so far has spoken in hex. Load an assembler's symbols and you can name things instead: `BREAK START` rather than `BREAK 0100` , `DUMP MSG/20` , `EXAMINE BD05` . A symbol is accepted anywhere an address is typed, and in a `BREAK ... IF` condition.

```
SYMBOLS LOAD prog.SYM          a symbol table
SYMBOLS LOAD ALTMON.PRN        ...or an assembler listing
BREAK START
DUMP MSG/20
BREAK 200 IF HL==STACK
SHOW SYMBOLS                   all of them
SHOW SYMBOLS SI0*              filtered by a glob
SYMBOLS CLEAR                  forget them
```

The same disassembly, with `ALTMON.PRN` loaded, reads symbolically. A symbol is accepted wherever a hex address was — so you can name the range — and the output names what it can. `DISASM` is the only

command whose *output* is symbolic: `DUMP` still prints hex and ASCII, since nothing in a data block says which bytes are an address and which are just bytes.

```
altairsim> SYMBOLS LOAD ALTMON.PRN
96 symbol(s) from ALTMON.PRN
altairsim> DISASM MONIT-F811
MONIT:
F800 3E 03    MVI A,03
F802 D3 10    OUT 10
F804 D3 12    OUT 12
F806 3E 11    MVI A,11
F808 D3 10    OUT 10
F80A D3 12    OUT 12
F80C 31 00 C0 LXI SP,SPTR
F80F CD A5 FB  CALL DSPMSG
```

`MONIT` resolves to `F800`, so the range starts where you named it — naming an address is *reference*, and that works everywhere an address is typed. Two more things happen on top of it. A program **label** heads its own line the way an assembler listing prints it: `MONIT:` sits above `F800`, so a jump destination announces itself where it lands. And a 16-bit **operand** reads as a name — `CALL FBA5` becomes `CALL DSPMSG`, and `LXI SP,C000` becomes `LXI SP,SPTR`.

The two use the symbol table differently, and the difference is deliberate. A leading label comes from **program labels only**: `SPTR` is an **EQU** (the listing marks it with an `=`), so it never *heads* a line — a constant that merely equals a code address must not masquerade as one, the same rule that keeps `0005` from printing a phantom `BD0S:` label. But an operand is a value the instruction *points at*, and there an **EQU** that is really an address is exactly what you want to read: so `LXI SP,SPTR` names its target even though `SPTR` is an **EQU**, and `CALL 0005` reads as `CALL BD0S` for the same reason. A real label wins when a label and an **EQU** share a value.

Only a 16-bit operand is treated as an address. A byte immediate stays a number — an `MVI A,03` is not turned into a symbol even if some **EQU** happens to equal three, because two hex digits are a count, not an address.

Try it. `examples/debugger/` is a 46-byte program built for exactly this — a `.PRN` to `SYMBOLS LOAD`, a `.HEX` to `LOAD`, and a `README` that walks you from a symbolic `DISASM` through single-stepping, breaking on a label, and running it until it prints. Every rule above is something you can watch happen there, including the **EQU**-address operand and the byte that stays a number.

Two kinds of file, and the toolchains that write them. A **.SYM** is a flat list of name = value. Two toolchains write one: Digital Research's `MAC / RMAC` assemblers (every symbol, read by `SID`), and — with the right switches — Microsoft's **L80** linker. **L80**'s `/M` prints a *map* to the console, but `filename/N/Y/E` writes a real **filename.SYM**; the catch is that an **L80** **.SYM** holds **globals only** (the `PUBLIC` names), so a

module's local labels and `EQU` s are not in it. For those, use the assembler's listing. A `.PRN` or `.LST` is the assembler's own listing — from CP/M `ASM` , Microsoft `M80` , or `MAC` — and it is the richer source, because it marks an `EQU` and so can tell a constant apart from a program label: only real labels are offered back as addresses, so `0005` never starts printing as `BDOS` .

Addresses must be absolute. A relocatable `M80` listing marks its addresses, and loading one is refused by the offending line — link it and load the `.SYM` , or assemble to an absolute origin. A `.SYM` is written after linking and is absolute already, so it never has this problem.

Symbols are yours, not the machine's. Like a breakpoint, the table is the debugger's view, not part of any board — it survives `RESET` , `POWER` , and `CONFIG LOAD` , and `SYMBOLS CLEAR` is its `NOBREAK` . Loading two files **merges** them (the newest of a clashing name wins, and the command says how many were redefined); `SYMBOLS LOAD <file> REPLACE` starts fresh. A machine file can name a symbol file in its `startup` , and `CONFIG SAVE` writes the filename back out — the file, not the parsed table, exactly as it does for a built-in ROM.

A name beats a hex literal. If a symbol is spelled like a number — `FACE` , `BEEF` — the symbol wins; write `0FACE` (or `$FACE`) to force the number, the same escape that tells the register `A` from the number `0A` .

Searching, filling, moving — `SEARCH` , `FILL` , `MOVE` , `COMPARE`

The block operations. `COMPARE` will take a file as its second operand, which is how you check what the machine loaded against what you meant to load.

```
SEARCH 0-FFFF C3 00 2C      find those bytes
SEARCH 0-FFFF "BDOS"       ...or that string
FILL 100-1FF 00
MOVE 100-1FF 2000
COMPARE 100-1FF 2000       ...or against a file
```

Running real bus cycles by hand — `IN` , `OUT`

These are not simulated reads. **`IN` runs an input cycle on the bus, with every side effect a real one would have** — it will consume a character from a UART's receiver, it will advance a disk controller's sector counter. That is the point of them: it is how you poke a board the way the guest's software would, without writing any guest software.

```
IN 10      run a real IN cycle on port 10
OUT FF 55  run a real OUT cycle
```

Asking without touching — WHO

WHO asks which **board** *would* answer — the one that decodes the address or port you name, reported by its board id. **No cycle is run and nothing is consumed.** It is the question you want when IN 10 gives you FF and you cannot tell whether that is data or whether nothing is there at all.

The console 2SIO decodes port 10, so it answers by name — and reads and writes can land on different boards, so WHO reports each:

```
altairsim> WHO IO 10
port 10 IN:  sio0
port 10 OUT: sio0
```

A port nothing decodes answers nobody, and now you know the FF was a floating bus, not data:

```
altairsim> WHO IO 20
port 20 IN:  nobody (an IN here reads FF)
port 20 OUT: nobody (an OUT here goes nowhere)
```

It reports contention, and it reports PHANTOM* — so if two boards are fighting, or if one board has switched another one off, WHO is where you find out. On the default machine the DBL boot ROM lives inside mem0 at FF00 and asserts PHANTOM* to shadow the RAM beneath it, so a read there comes from the ROM while a write falls through to nowhere — and WHO flags both:

```
altairsim> WHO FF00
FF00 read  mem0  [PHANTOM* asserted]
FF00 write nobody -- floats to FF (a write here is simply gone) [PHANTOM* asserted]
```

WHO <addr> asks about a memory address; WHO IO <port> asks about a port.

Looking at the bus itself — SHOW BUS

Where WHO asks about one address, SHOW BUS shows you the whole backplane at once.

SHOW BUS IRQ is the only window onto the interrupt wiring, and interrupt wiring is the part of a machine you cannot see. A board strapped to a line that nothing listens to fails in total silence — the software just never gets its interrupt, and there is nothing to look at. This command is what makes that visible.

`SHOW BUS CONTENTION` is the one to reach for when a machine you built yourself is misbehaving for no reason. Two boards decoding the same port is a real hardware fault, and the simulator will not quietly pick a winner for you.

<code>SHOW BUS MAP</code>	who decodes what in memory – and what floats
<code>SHOW BUS IO</code>	who decodes which ports
<code>SHOW BUS IRQ</code>	the eight interrupt lines: who is strapped where, who is pulling
<code>SHOW BUS CONTENTION</code>	where two boards answer the same thing

The machine over time — `TRACE`, `HISTORY`

`WHO` and `SHOW BUS` are snapshots — the backplane as it is *now*. `REGS` and `STEP` are the machine as it is *now*. `TRACE` and `HISTORY` show you all of that over *time*, which is what you want when the bug is not where the machine stopped but somewhere in how it got there.

`HISTORY` is a flight recorder. A fixed-size ring is always filling while the machine runs, so when a breakpoint fires — or the machine wanders off into the weeds — the run-up to it is *already* recorded. You do not arm it; it is on. A bare `HISTORY` shows the last sixteen **instructions**, oldest first; `HISTORY <n>` shows the last *n*. Each line is exactly what `STEP` prints — the registers and flags as the machine stood, and the decoded mnemonic it was about to run — so the recording reads like a `STEP` you did not have to be there for. The mnemonic is decoded from the bytes that *actually ran* at that address, so self-modifying code reads truthfully.

There is a second recorder underneath, for when the CPU view is not enough: **`HISTORY BUS`** is the raw bus cycles — no registers and no mnemonics, just `T-STATE`, `TYPE`, `ADDR` and `DATA`, and then **who drove the cycle and who answered it**. The processor drives most cycles, so that column reads `cpu`; a **DMA transfer names the board** that stole the bus instead (a DMA move originates no instruction, so it shows up here and not in the CPU view). The *answered* column is the board that decoded the address — or `--` when nobody did and the read floated to `FF`. So a guest poking a port that no board decodes reads `cpu - > --`, and a DMA controller filling a framebuffer reads `dazzler -> mem0`. `HISTORY CPU` names the default out loud.

<code>HISTORY</code>	the last 16 instructions
<code>HISTORY 100</code>	the last hundred (a count, so decimal)
<code>HISTORY BUS</code>	the last 16 bus cycles instead
<code>HISTORY BUS 100</code>	the last hundred cycles

`TRACE` logs every cycle as it happens — to the console, or to a file. Like the breakpoints that watch the bus, it is not a CPU feature: it watches the same stream every board sees, so it works unchanged on any processor you put in the machine.

A **MASK** narrows the log to the kinds of cycle you name. With no mask every cycle is logged; with one, a cycle is kept if it matches *any* kind you listed. There are five:

- **IN** — an I/O read, an **IN** from a port.
- **OUT** — an I/O write, an **OUT** to a port.
- **IRQ** — an interrupt-acknowledge cycle: the **INTA** the CPU runs to take the interrupting device's instruction off the bus.
- **DMA** — every cycle a granted bus master drove, *whatever its type*. **MASK=DMA** is the whole transfer a board stole the bus for, not just its reads or its writes.
- **CONTENTION** — a cycle more than one board answered: the bus fact that two boards decoded the same address at once (the same fault **SHOW BUS CONTENTION** reports, caught as it happens).

```
TRACE ON           every cycle, to the console
TRACE ON run.log    ...to a file instead
TRACE ON MASK=IN,OUT just the port traffic
TRACE ON MASK=IRQ,DMA just interrupts and DMA
TRACE OFF          stop tracing
```

Tracepoints — tracing one part of a program

A whole-program trace is a firehose. A mask narrows it by *category*, but often you do not want a category — you want a *place*: this subroutine, and nothing else.

That is a tracepoint. Add **TRACE ON** or **TRACE OFF** to a **BREAK** and it stops being a breakpoint: instead of stopping the machine it flips the trace, and the machine runs on. Two of them bracket a region.

```
altairsim> BREAK 2C00 TRACE ON    start tracing when PC reaches 2C00
altairsim> BREAK 2C40 TRACE OFF    ...and stop again at 2C40
altairsim> RUN FF00
```

TRACE ON traces the instruction *at* its address; **TRACE OFF** does not. The region is **[on, off)** — exactly the half-open range you would write down if someone asked you which instructions were in the subroutine.

A trace toggle works on the bus kinds the same way **IF** now does, and the cycle that triggered it is the *first line* of the trace, not the line above it: a trace should show its own reason.

```
altairsim> BREAK MEM W 2000 TRACE ON    trace onward from whatever writes 2000
```

They compose with **IF**, and they still do not stop:

```
altairsim> BREAK 200 IF HL==8000 TRACE ON
```

Where the trace goes is **TRACE** 's business, not the tracepoint's. A tracepoint that has never been told anything traces to the console. To send it to a file, configure it first — and this is what **TRACE OFF** is for: it stops the tracing but *remembers where it was going*, so a tracepoint can pick it up again.

```
altairsim> TRACE ON run.log MASK=DMA    configure it: file, mask – and it starts
altairsim> TRACE OFF                    stop, but the file and mask are remembered
altairsim> BREAK 2C00 TRACE ON          arm the region
altairsim> BREAK 2C40 TRACE OFF
altairsim> RUN FF00                     run.log gets the DMA cycles, from 2C00 to 2C40, and nothing else
```

Tracepoints appear in **BREAK** 's listing with the rest, and their **hits** count the times they fired. A tracepoint never hides an ordinary breakpoint at the same address: if both are set, the trace flips *and* the machine stops.

What a board is doing — **SET ... DEBUG** , **SHOW DEBUG**

TRACE and **HISTORY** watch the *bus* — the cycles, addresses and data every board shares. Some parts of the machine can also narrate what they are doing in their *own* terms: a floppy controller stepping the heads and reading a sector, a serial chip taking a byte, a socket answering a call. That is what the **diagnostic channels** are for. Each instrumented part has a named channel with a handful of flags, and you switch on the ones you want to hear about.

SHOW DEBUG lists every channel there is, its flags, and where the output is going:

```
altairsim> SHOW DEBUG
debug  (runtime diagnostics -- the sink and flags do not survive CONFIG SAVE)

sink  stderr  -- SET CONSOLE DEBUG=stderr|stdout|<file>

CHANNEL  FLAGS  (an enabled flag is UPPER-CASE)
-----  -----
dsk0     SECTOR seek
6850     serial
socket   connect

SET <channel> DEBUG=<flag>[,<flag>] enables; NODEBUG=<flag> disables;
DEBUG=all / DEBUG=none turn every flag on / off.
```

A flag printed in capitals is on. Turn one on with `DEBUG=`, off with `NODEBUG=`; both are additive, take a comma-separated list, and understand `all` and `none`:

```
SET dsk0 DEBUG=sector,seek    narrate both sector reads and head seeks
SET dsk0 NODEBUG=seek         quiet the seeks, keep the sector reads
SET dsk0 DEBUG=all            everything this board can say
SET dsk0 NODEBUG=all          silence it
```

The name in front of `DEBUG` is the channel. Usually it is a board's id — `dsk0` above — but a shared chip or the socket layer has one too (`6850`, `socket`), so the same switch reaches parts of the machine that are not boards at all. An unknown flag is refused and *nothing* changes: the switch is all-or-nothing, so a typo in a list never leaves half of it applied.

Every line names its channel and is prefixed with **the PC of the instruction that drove it**, so a diagnostic line points straight at the code working the board:

```
2C38 dsk0: sector drive=0 track=0 sector=1
007F dsk0: seek drive=0 track=0 -> 1
```

At the monitor prompt — with the machine stopped — there is no such instruction, and the column reads `----`.

Where the output goes is one setting for the whole facility, aimed with `SET CONSOLE DEBUG=`:

```
SET CONSOLE DEBUG=stderr      the default
SET CONSOLE DEBUG=stdout
SET CONSOLE DEBUG=trace.log    append to a file
```

Tab completes all of it — the channel names after `SET`, `DEBUG` / `NODEBUG` after the channel, and the flag values (with `all` and `none`) after the `=`.

None of this is saved by `CONFIG SAVE`. A diagnostic is something you switch on to watch a problem, not a property of the machine, so a config you write while debugging does not carry the noise into every later run.

A copy of the session — SET CONSOLE log

TRACE records the bus and DEBUG records what a board narrates; SET CONSOLE log records the *terminal* — everything you saw. Guest output and the keys you typed both go to a host file as they happen, so you can read a whole session back later, or hand it to someone who was not there.

```
SET CONSOLE log=session.txt      start copying the session to a file
SET CONSOLE log=off              stop (an empty path does the same)
```

The file is opened for *append*: pointing log at the same file twice in a session adds to it rather than erasing what you already caught. Like TRACE and DEBUG it is a diagnostic and not part of the machine, so CONFIG SAVE does not carry it — a config written while you are capturing a session does not turn logging on for every later run.

A debugging session

Here is the whole toolset in one short session on the altmon machine. ALTMON prints its ALTMON 1.3 banner at reset, and it does it without passing a pointer to its print routine — a neat trick worth taking apart. Break at the sign-on routine FBA5 and run from the reset entry:

```
altairsim> BREAK FBA5
breakpoint 1: pc      FBA5
altairsim> RUN F800

breakpoint 1 (pc      FBA5) -- stopped at FBA5
8 instructions, 81 T-states.
C0Z0M0E0I1 A=11 BC=0000 DE=0000 HL=F81F SP=BFFE IE=0 PC=FBA5 POP H
```

The breakpoint prints the registers for you — no `REGS` needed. The PC is on `POP H`, the first instruction of the routine. Disassemble to see the whole thing:

```
altairsim> DISASM
FBA5 E1      POP H
FBA6 7E      MOV A,M
FBA7 CD 48 FB CALL FB48
FBAA B6      ORA M
FBAB 23      INX H
FBAC F2 A6 FB JP FBA6
FBAF CD 46 FB CALL FB46
FBB2 E9      PCHL
FBB3 CD BC FB CALL FBBC
FBB6 FE 1B   CPI 1B
FBB8 C8      RZ
FBB9 C3 48 FB JMP FB48
FBBC DB 10   IN 10
FBBE 0F      RRC
FBBF D2 BC FB JNC FBBC
FBC2 DB 11   IN 11
```

There is the trick: `POP H` takes the routine's *own return address* into `HL`. The message is stored inline, right after the `CALL FBA5` — so `HL` now points at it. Then `MOV A,M` fetches a character, `CALL FB48` prints it, `ORA M` tests the byte, `INX H` advances, and `JP FBA6` loops. Step into it and watch the banner come out a character at a time:

```
altairsim> STEP 20
C0Z0M0E0I1 A=11 BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBA6 MOV A,M
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBA7 CALL FB48
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB48 PUSH PSW
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB49 IN 10
C0Z0M0E0I1 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB4B ANI 02
C0Z0M0E0I0 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB4D JZ FB49
C0Z0M0E0I0 A=02 BC=0000 DE=0000 HL=F812 SP=BFFC IE=0 PC=FB50 POP PSW
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB51 ANI 7F
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB53 OUT 11
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=BFFE IE=0 PC=FB55 RET
C0Z0M0E0I1 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBAA ORA M
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F812 SP=C000 IE=0 PC=FBAB INX H
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBAC JP FBA6
C0Z0M0E0I0 A=0D BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBA6 MOV A,M
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=C000 IE=0 PC=FBA7 CALL FB48
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=BFFE IE=0 PC=FB48 PUSH PSW
C0Z0M0E0I0 A=0A BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB49 IN 10
C0Z0M0E0I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB4B ANI 02
C0Z1M0E1I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB4D JZ FB49
C0Z1M0E1I0 A=00 BC=0000 DE=0000 HL=F813 SP=BFFC IE=0 PC=FB49 IN 10
```

`POP H` landed `HL` on `F812`, the byte right after the `CALL`, and the first character is `0D` — a carriage return. `FB48` is the console-output routine: it polls the 2SIO status (`IN 10`, `ANI 02`) until the transmitter is ready, then `OUT 11` sends the byte and returns. Back in the loop, `INX H` steps `HL` to `F813` and the next character `0A` (line feed) follows the same path — and so on, until a byte with its high bit set marks the end of the string. Twenty steps in, you have watched the first two characters of the banner reach the terminal.

Saving state — `SNAPSHOT`, `RESTORE`

`SNAPSHOT` writes the machine's whole state — the CPU, the clock, and every board's registers, RAM and latches — to a file, and `RESTORE` reads it back into a machine of the same shape. So you can save the machine at a moment and return to it later.

Mark a spot, change something, and put it back:

```
altairsim> SNAPSHOT before.snap
snapshot written to before.snap
altairsim> DEPOSIT 100 00 00 00 00      trample four bytes
altairsim> RESTORE before.snap
restored from before.snap
altairsim> DUMP 100-103                  they are back
0100  DE AD BE EF                      ....
```

It saves *state*, not *configuration* — the boards themselves are not in the file. `RESTORE` loads back into the machine you already have, and refuses a file that does not match its shape (the same boards, ids, and order), leaving the running machine untouched. Build that shape with the machine file or a `CONFIG LOAD` first, then restore into it.

Getting an assistant to do it — the MCP server

Everything in this document is a command you type. It is also a tool an **AI assistant** can call. Start the machine with `--mcp` instead of at a terminal —

```
$ altairsim <machine> --mcp
```

— and the same debugger is offered to an assistant as structured tools: it can set a breakpoint, run to it, read the registers, disassemble, step, dump memory, and read the bus recorder, on the *same* machine object the monitor drives. So instead of learning the commands, you can describe the symptom in a sentence and let the assistant work the machine for you. (The MCP server is covered in full in the **MCP server** chapter of the User Manual, including how to register it with clients other than the one below.)

Setting it up — the example is Claude Code. Two one-time steps. First, register the server so the assistant can reach the machine; run this in the directory that holds your machine file, so the relative path and the host bridge both resolve there:

```
$ claude mcp add altairsim -- altairsim <machine> --mcp
$ claude mcp list          # confirm it registered and is reachable
```

Second, hand the assistant its briefing. `DRIVING-WITH-AI.md` ships in the package; it is written for the assistant, not for you — it teaches these tools and the recipes for booting, building and debugging over them. Copy it into the same directory, start `claude` there, and point the assistant at it before you give it a job:

```
$ cp /path/to/DRIVING-WITH-AI.md .
$ claude
> Read DRIVING-WITH-AI.md, then use the altairsim MCP tools for what follows.
```

From then on you talk to the assistant, not to the server.

What you say. Give the assistant the whole job in plain language, and name the tools so it drives the simulator rather than guessing:

Using the altairsim MCP tools, boot the machine and run `HELLO.COM`. It should print `HELLO, WORLD` but it prints `ELLO, WORLD` — find the bug and fix the source.

My loader hangs instead of reaching the prompt. Boot with the MCP tools, break at `2C00` where it relocates itself, and single-step from there to tell me where it goes wrong.

Something is overwriting the BIOS at `E400`. Set a memory-write breakpoint there, run until it trips, and show me the instruction and registers that did it.

What the assistant does with that. It works the same loop you would, one tool call at a time. Given the first sentence, it boots the machine, runs the program, and sees `ELLO, WORLD`; sets a breakpoint at `0100`, runs there, and disassembles — where it finds the pointer advanced by an `INX H` before the first character is read, so the `H` is skipped. It corrects the source, reassembles it on the CP/M disk, and runs again to confirm `HELLO, WORLD`. You watch the reasoning and the fix; you type none of the commands.

When a typed tool does not reach. The assistant is not limited to the structured tools: it can run any monitor command and read the reply, so a conditional breakpoint (`BREAK 200 IF HL==8000`) or an octal dump is one call away, exactly as it is for you.

`examples/ai-mcp/` is a ready-made version of the first sentence above — a CP/M machine and a `HELLO.ASM` with one deliberate bug — with a walkthrough of the session the assistant runs to find and fix it. It is the fastest way to see this work end to end.

Things to know about the bus

Not commands — facts about how the backplane behaves, the kind that turn a baffling reading into an expected one.

When `FF` is not data

`FF` is a perfectly good byte — a `RST 38`, an `RST 7`, the value -1 — so most of the time it reads back as exactly what a board put there. But it is also what the bus gives you when *nobody answered*, and telling those two apart is worth a habit.

An `IN` from a port nothing decodes, or a read from an address no board answers for, returns `FF`. That is not an error code and it is not a convention we invented — it is what a **floating bus** reads. Nobody is driving the data lines, they idle high, and the processor faithfully reads eight ones. A real Altair does exactly this.

It has a famous consequence, and it is worth knowing because you will meet it: on a machine with no interrupt-vector board, a board pulls the interrupt line, nobody drives the data bus during the acknowledge cycle, the processor reads `FF` — and `FF` is `RST 7`. That is not a fallback anybody coded. It is what the hardware does, and it is why the interrupt vector on a bare Altair is `RST 7`.

So when a read gives you `FF` you did not expect, ask `WHO`: it tells you whether a board answered with that byte or the bus floated because none did.